FastBuyWave

Container Documentation

3-Container Architecture · Podman on Linux · No Docker Compose

3	767	131K+	~90
CONTAINERS	PROJECT FILES	LINES OF CODE	GIT COMMITS

Technology Stack

Angular · Node.js · Spring Boot · Maven · Python · MySQL · Podman

Architecture Overview

Three independent containers, each built and run separately with Podman on Linux. No Docker Compose or orchestration layer — each service can be updated or restarted independently.

Container	Image Name	Port	Technology
Angular Frontend	fastbuy-frontend	4200	Node 20 · SSR Express
Spring Boot Backend	fastbuy-backend	8080	Maven 3.9 · JRE 21 Alpine
Python Email Server	fastbuy-email-server	1234 / 8025	Python 3.12-slim

1. Angular Frontend

angular_fe/Dockerfile · Node 20 · SSR Express · Single-stage build

Runtime: Node 20 — official Node.js LTS. Single-stage build: my-lib + angular_fe copied together. Pre-built dist is copied as-is — ng build is NOT executed inside the container. Environment files generated at build time via RUN printf.

Dockerfile

```
FROM node:20 AS build
WORKDIR /app
COPY my-lib ./my-lib
COPY angular_fe ./angular_fe

# Generate environment files at build time
RUN mkdir -p /app/angular_fe/src/app/environments
RUN printf 'export const environment = {\n  production: false,\n  backendPort: 8080,\n
ssrBackendHost: "localhost",\n  hostMapping: {} as Record<string, string>,\n
forceBackendHost: null as string | null,\n};\n' >
/app/angular_fe/src/app/environments/environment.ts
RUN printf 'export const environment = {\n  production: true,\n  backendPort: 8080,\n
ssrBackendHost: "localhost",\n  hostMapping: {} as Record<string, string>,\n
forceBackendHost: null as string | null,\n};\n' >
/app/angular_fe/src/app/environments/environment.prod.ts

# Full build pipeline (commented, kept for reference):
# RUN apt-get update && apt-get install -y build-essential python3 make g++
# RUN cd my-lib && rm -rf node_modules package-lock.json && npm install && npm run build
# RUN cd angular_fe && rm -rf node_modules package-lock.json && npm install
# RUN cd angular_fe && npm install @angular/localize@18
# RUN cd angular_fe && npx ng build --configuration=production

WORKDIR /app/angular_fe/dist/angular_fe/server
ENTRYPOINT ["node", "server.mjs"]
```

How the path works

On your Linux system, from the angular_fe/ root:

```
node dist/angular_fe/server/server.mjs
```

Inside the container after COPY angular_fe ./angular_fe, that same folder lands at:

```
/app/angular_fe/dist/angular_fe/server/
```

The Dockerfile sets WORKDIR to that exact path, then ENTRYPOINT runs node server.mjs. /app exists only inside the container, not on your filesystem.

Details

Aspect	Detail
Base image	node:20 — official Node.js LTS
Strategy	Single-stage — my-lib + angular_fe copied together
Environment files	Generated via RUN printf at build time — not in repo
Entry point	node server.mjs — run from WORKDIR /app/angular_fe/dist/angular_fe/server
Pre-built dist	Dockerfile skips ng build — dist/ must exist before running podman build
Build context	Run podman build from project root — my-lib/ must be in scope

2. Python Email Server

ServerEmailPython/Dockerfile · Python 3.12-slim · Single-stage build

Lightweight single-stage build using python:3.12-slim to minimize image size. Copies all sources, installs dependencies from requirements.txt, marks start.sh as executable. Entry point starts the SMTP relay directly via python server.py.

Dockerfile

```
FROM docker.io/python:3.12-slim AS build
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
RUN chmod +x /app/start.sh
ENTRYPOINT ["python", "server.py"]
# ENTRYPOINT ["./start.sh"]   <- alternative shell-script startup
```

Details

Aspect	Detail
Base image	python:3.12-slim — lightweight, minimizes final image size
Strategy	Single-stage — copy all sources, pip install
Entry point	python server.py — starts the SMTP relay directly
Alternative	./start.sh — available via commented ENTRYPOINT line
Dependencies	aiosmtpd, google-auth, google-api-python-client, requests, python-dotenv, Jinja2, etc.

3. Spring Boot Backend

springboot_be/Dockerfile · Maven 3.9 · JRE 21 Alpine · Multi-stage build

Multi-stage build: Stage 1 uses the full Maven + JDK image to compile the JAR. Stage 2 uses only a minimal Alpine JRE — the final image contains no Maven, no source code, no build tools. Lean and secure production image.

Dockerfile

```
# Stage 1 — Build
FROM docker.io/library/maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2 — Runtime
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Details

Aspect	Detail
Stage 1 — Build	maven:3.9-eclipse-temurin-21 · runs mvn clean package -DskipTests
Stage 2 — Runtime	eclipse-temurin:21-jre-alpine · minimal Alpine JRE only
Advantage	Final image = JRE + JAR only. No Maven, no sources, no build tools
Exposed port	8080
Entry point	java -jar app.jar

4. Podman Commands

Linux · Independent container management · No Compose

Build

Container	Command	Notes
Angular Frontend	`podman build -t fastbuy-frontend .`	Run from project root (not from angular_fe/)
Spring Boot Backend	`podman build -t fastbuy-backend .`	Run from springboot_be/
Python Email Server	`podman build -t fastbuy-email-server .`	Run from ServerEmailPython/

Run

Container	Command	Notes
Angular Frontend	`podman run -it --rm -p 4200:4200 fastbuy-frontend /bin/bash`	Interactive, auto-removed on exit
Spring Boot Backend	`podman run -it --rm -p 8080:8080 fastbuy-backend /bin/bash`	Interactive, auto-removed on exit
Python Email Server	`podman run -it --rm -p 8025:8025 fastbuy-email-server /bin/bash`	Interactive, auto-removed on exit

Remove Images

Target	Command	Notes
Frontend	`podman rmi -f fastbuy-frontend`	By name
Backend	`podman rmi -f fastbuy-backend`	By name
Email server	`podman rmi -f fastbuy-email-server`	By name
Any image	`podman rmi -f <IMAGE_ID>`	By hash ID

---

5. Project Structure

```
├── angular_fe/
│   ├── Dockerfile
│   └── ...
├── springboot_be/
│   ├── Dockerfile
│   └── ...
├── ServerEmailPython/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── start.sh
│   └── ...
└── my-lib/
    └── ...
```

6. Important Notes

Topic	Detail
No Docker Compose	All containers built and run independently with Podman on Linux
Angular build context	Run podman build from the project root — my-lib/ must be reachable by COPY
Pre-built dist	Dockerfile does not run ng build — make sure dist/ exists before podman build
/app in container	/app is the WORKDIR inside the container only — does not exist on your filesystem
Environment files	Generated at build time via RUN printf — do not commit to repo
start.sh permissions	chmod +x handled inside the Python Dockerfile
Docker Hub	All three images also available on Docker Hub for direct pull